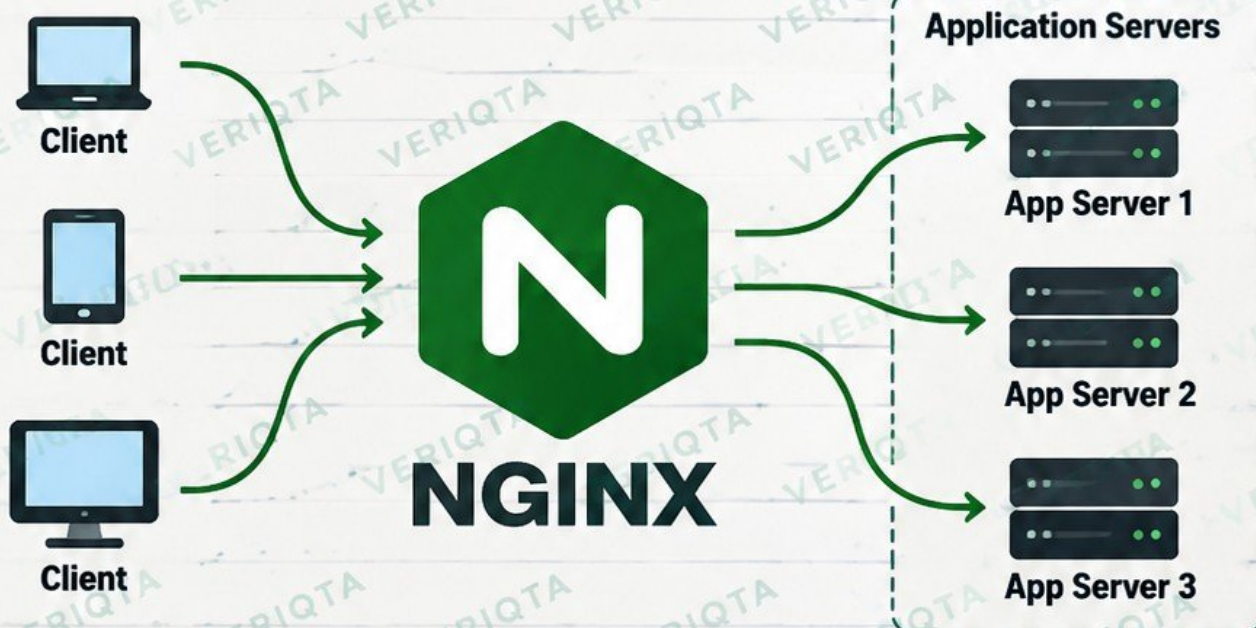


NGINX

Crash Course

✓ Concepts | ✓ Configuration | ✓ Real Examples | ✓ Troubleshooting



Web
Server



Reverse
Proxy



Load
Balancer



Security
with HTTPS



Logging &
Troubleshooting



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

1. NGINX: What It Is and Why DevOps Engineers Use It

What is NGINX?

- NGINX is a high-performance web server that can also act as a reverse proxy, load balancer, and HTTP cache.
- It receives client requests and forwards them to the correct destination.
- It is lightweight, fast, and used in millions of production environments.

Why DevOps Engineers Use NGINX

- Serve static websites and web applications.
- Act as a reverse proxy for application servers (Node.js, Python, Java, etc).
- Distribute traffic across multiple servers (load balancing).
- Improve performance with caching.
- Provide SSL/TLS termination (HTTPS).
- Enhance security and control access.

Key Roles of NGINX



Web Server

Serves static files like HTML, CSS, JavaScript, and images to clients.



Reverse Proxy

Forwards client requests to backend servers and hides the application server from the internet.



Load Balancer

Distributes traffic across multiple backend servers for high availability and better performance.



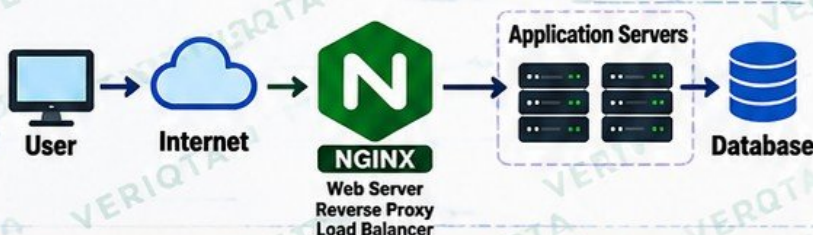
Cache

Stores frequently requested content to reduce load on backend servers and improve speed.

How NGINX Works (Request Flow)



Where NGINX Sits in a Real Architecture



Common Real-World Use Cases

- ✓ Hosting a company website
- ✓ Running multiple application servers behind a load balancer
- ✓ Exposing a local application to the internet
- ✓ Terminating SSL/TLS certificates
- ✓ Improving performance with caching
- ✓ Handling high traffic in production



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

2. Install NGINX and Understand the Service

1. Update Your System

It is a good practice to update your package list before installing new software.

```
sudo apt update
sudo apt upgrade -y
```

3. Verify the Installation

Check the installed version.

```
nginx -v
```



```
nginx version: nginx/1.24.0 (Ubuntu)
```

5. Common Service Commands

Use systemctl to manage NGINX.

Action	Command
Check status	<code>systemctl status nginx</code>
Start NGINX	<code>sudo systemctl start nginx</code>
Stop NGINX	<code>sudo systemctl stop nginx</code>
Restart NGINX	<code>sudo systemctl restart nginx</code>
Reload configuration	<code>sudo systemctl reload nginx</code>
Enable on boot	<code>sudo systemctl enable nginx</code>
Disable on boot	<code>sudo systemctl disable nginx</code>

7. Access the Default Page

Open your server's IP address in a browser.

```
http://<your-server-ip>
```



If you are on the server itself, you can also use:
<http://localhost>

2. Install NGINX

Install the NGINX web server on Ubuntu.

```
sudo apt install nginx -y
```



NGINX will be installed along with required dependencies.

4. Understand the Service

NGINX runs as a systemd service on Ubuntu. Use systemctl to manage it.

```
systemctl status nginx
```

```
nginx.service - A high performance web server
Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
Active: active (running) since Thu 2026-10-01 10:20:15 UTC
Main PID: 1234 (nginx)
Tasks: 3 (limit: 2345)
Memory: 5.2M
```

6. Verify NGINX is Running

After installation, NGINX should be running. Check with systemctl or using a web browser.

```
sudo systemctl status nginx
```



» Next Step

Now that NGINX is installed and running, the next page will explore the NGINX configuration structure and important files.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

3. Inside the NGINX Configuration

1. Main Configuration File

The main configuration file is:

`/etc/nginx/nginx.conf`

- This file contains the global settings.
- It includes other configuration files from `conf.d` or `sites-enabled`.
- You usually do not put website configs directly here.

3. How Files Are Included

The main `nginx.conf` file includes other files using the `include` directive.

```
# /etc/nginx/nginx.conf (simplified)
http {
    include /etc/nginx/mime.types;
    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}
```

2. Important Directories

NGINX configuration files are organized in these directories.

/etc/nginx/	
<code>nginx.conf</code>	Main configuration file
<code>conf.d/</code>	Additional configuration files
<code>sites-available/</code>	Individual site configs (not enabled yet)
<code>sites-enabled/</code>	Active site configs (symlinks from sites-available)
<code>snippets/</code>	Reusable configuration blocks

- This tells NGINX to load all `.conf` files from these directories.
- This structure helps you separate and manage multiple websites easily.

4. NGINX Configuration Hierarchy

NGINX configuration is organized in a hierarchy: `http` → `server` → `location`.

http (Global Settings)

- Top-level block
- Applies to all servers
- Defines global settings like log format, MIME types, and default settings

server (Virtual Host)

- Defines a website or domain
- Listens on a specific port and domain
- Can have multiple server blocks

location (URL Paths)

- Handles requests for specific URL paths
- Can have multiple location blocks
- Used for routing, proxying, serving files, etc.

5. Example NGINX Configuration

A simple example showing the `http` → `server` → `location` structure.

```
http {
    server {
        listen 80;
        server_name example.com;
        root /var/www/html;
        index index.html;

        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

http block
(global settings)

server block
(website configuration)

location block
(URL routing)

6. Common File Types and Purpose

File / Directory	Purpose
<code>nginx.conf</code>	Main configuration file
<code>conf.d/</code>	Additional <code>.conf</code> files (often used on some distributions)
<code>sites-available/</code>	Site configurations (not active)
<code>sites-enabled/</code>	Active site configurations (symlinks)
<code>snippets/</code>	Reusable configuration blocks
<code>mime.types</code>	Defines file types and MIME types
<code>fastcgi_params</code>	Default FastCGI parameters
<code>proxy_params</code>	Default proxy parameters



Junior Engineer Tip

Keep your main `nginx.conf` clean and place each website in a separate file inside `sites-available`, then enable it with a symlink in `sites-enabled`.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

4. Your First NGINX Server Block

1. Create a Website Directory

Create a folder for your website and add a simple HTML file.

```
sudo mkdir -p /var/www/my-site
echo "<h1>Welcome to my site!</h1>" |
sudo tee /var/www/my-site/index.html
sudo chown -R www-data:www-data
/var/www/my-site
```

3. Understand the Important Directives

Directive	What It Does
<code>listen 80;</code>	Tells NGINX to listen on port 80 for HTTP traffic.
<code>server_name my-site.com;</code>	The domain name for this website. Use <code>_</code> for testing without a domain.
<code>root /var/www/my-site;</code>	The directory where your website files are stored.
<code>index index.html;</code>	The default file to serve when a user visits the root URL.
<code>location / { ... }</code>	Defines how requests to <code>/</code> are handled.

4. Enable the Configuration

Create a symbolic link to enable the site.

```
sudo ln -s /etc/nginx/sites-available/
my-site.conf /etc/nginx/sites-enabled/
```

6. Reload NGINX

Apply the changes without stopping the service.

```
sudo systemctl reload nginx
```



This reloads the configuration and keeps NGINX running.

8. Check the Service Status (Optional)

Verify that NGINX is running.

```
sudo systemctl status nginx
```

2. Create the NGINX Server Block

Create a configuration file for your website.

```
sudo nano /etc/nginx/sites-available/my-site.conf
```

```
server {
    listen 80;
    server_name my-site.com;
    root /var/www/my-site;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```



Junior Engineer Tip

For testing on your server without a domain, use an underscore:

```
server_name _;
```

5. Test the NGINX Configuration

Check for syntax errors before reloading.

```
sudo nginx -t
```



You should see:
syntax is ok
test is successful

7. Test the Website

Open your server's IP address or domain in a browser.

```
http://<your-server-ip>
```

You should see:



Common Issues

- Permission issues on the website directory
- Syntax errors in the configuration
- Port 80 already in use
- Firewall blocking HTTP traffic (port 80)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

5. Serving Static Websites with NGINX

1. What Are Static Websites?

- Static websites are made of files like HTML, CSS, JavaScript, and images.
- NGINX serves these files directly to the browser.
- No application server is required.
- It is fast, lightweight, and commonly used for company websites, documentation sites, and simple web applications.

3. Create a Simple Static Website

Create the files and folders.

```
sudo mkdir -p /var/www/my-site/{css,js,images}
sudo nano /var/www/my-site/index.html
sudo nano /var/www/my-site/css/style.css
sudo nano /var/www/my-site/js/app.js
sudo cp /path/to/logo.png /var/www/my-site/images/
```

5. Set File Permissions

Give the correct permissions so NGINX can read the files.

```
sudo chown -R www-data:www-data /var/www/my-site
sudo chmod -R 755 /var/www/my-site
```

7. Reload NGINX and Test

Check the configuration and reload NGINX.

```
sudo nginx -t
sudo systemctl reload nginx
```

- ✓ Open your server's IP address or domain in a browser to view your website.

8. How a Browser Requests a Static File



2. Example Project Structure



4. NGINX Server Block Configuration

Configure NGINX to serve the static files.

```
server {
    listen 80;
    server_name my-site.com;
    root /var/www/my-site;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

6. Supported MIME Types

NGINX uses MIME types to tell the browser the file type.

File Type	Extension	MIME Type (Content-Type)
HTML	.html	text/html
CSS	.css	text/css
JavaScript	.js	application/javascript
Image (PNG)	.png	image/png
Image (JPG)	.jpg	image/jpeg
Image (SVG)	.svg	image/svg+xml

9. What Happens in the Background?

- 1 Browser sends an HTTP request for a file.
- 2 NGINX checks the server block and document root.
- 3 It finds the requested file on disk.
- 4 NGINX uses the correct MIME type.
- 5 It sends the file back to the browser.
- 6 The browser renders the HTML, loads CSS, JavaScript, and images, and displays the page.

Junior Engineer Tip

Always keep your static website files in a dedicated directory such as `/var/www/your-site` for better organization and security.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

6. Understanding location Blocks

1. What are location Blocks?

- The location block is used to match a requested URL and define how NGINX should handle it.
- You can serve static files, proxy requests, redirect, or apply special rules based on the URL.
- You can have multiple location blocks in a server block.

3. Basic Syntax

A location block is defined inside a server block.

```
server {
    listen 80;
    server_name example.com;

    location / {
        # configuration here
    }
}
```

5. Practical Examples

Common location blocks and their use cases.

Use Case	Location Block	Explanation
Serve main website	<code>location / { root /var/www/html; index index.html; }</code>	Handles all requests that do not match any other location.
Serve static images	<code>location /images/ { root /var/www/html; }</code>	Matches any URL starting with <code>/images/</code> .
Exact match for a page	<code>location = /about { root /var/www/html; }</code>	Matches only <code>/about</code> .
Serve PHP files	<code>location ~ \.php\$ { proxy_pass http://php-backend; }</code>	Matches any URL ending with <code>.php</code> (case-sensitive).
Serve image files (case-insensitive)	<code>location ~* \.(jpg jpeg png gif)\$ { expires 30d; }</code>	Matches image files with any case.
Proxy API requests	<code>location /api/ { proxy_pass http://api-backend; }</code>	Proxies all requests starting with <code>/api/</code> .

6. Real-World URL Routing Example

A more complete example with multiple location blocks.

```
server {
    listen 80;
    server_name example.com;

    location = / {
        root /var/www/html;
        index index.html;
    }

    location /images/ {
        root /var/www/html;
        expires 30d;
    }

    location ~ \.php$ {
        proxy_pass http://php-app:9000;
        include fastcgi_params;
    }

    location /api/ {
        proxy_pass http://api-app:3000;
    }
}
```

2. Types of location Matches

Type	Syntax	Description	Example
Exact match	<code>= /path</code>	Matches the exact URL only.	<code>location = /about { ... }</code>
Prefix match	<code>/path</code>	Matches the URL starting with <code>/path</code> .	<code>location /images/ { ... }</code>
Regex match (case-sensitive)	<code>~ pattern</code>	Matches using regular expressions (case-sensitive).	<code>location ~ \.php\$ { ... }</code>
Regex match (case-insensitive)	<code>~* pattern</code>	Matches using regular expressions (case-insensitive).	<code>location ~* \.jpg\$ { ... }</code>
Named location	<code>@name</code>	Used for internal redirects.	<code>location @fallback { ... }</code>

4. Matching Precedence (Order)

NGINX checks location blocks in the following order:

- 1 Exact match `= /path` (highest priority)
- 2 Longest prefix match `/path/`
- 3 First regex match (in the order they appear) `~` or `~*`
- 4 If no match, use the generic prefix `/` (lowest priority)

7. How Matching Works (Example Requests)

See which location block will match each URL.

Request URL	Matched Location	Reason
<code>/</code>	<code>location = /</code>	Exact match
<code>/about</code>	<code>location = /about</code>	Exact match
<code>/images/logo.png</code>	<code>location /images/</code>	Longest prefix match
<code>/api/users</code>	<code>location /api/</code>	Prefix match
<code>/app.php</code>	<code>location ~ \.php\$</code>	Regex match
<code>/IMAGE/LOGO.PNG</code>	<code>location ~* \.(jpg jpeg png gif)\$</code>	Case insensitive regex
<code>/contact</code>	<code>location /</code>	Falls back to generic prefix



Key Takeaways

- Use the correct match type based on your use case.
- Exact match has the highest priority.
- Order matters for regex locations.
- Location blocks can be used for serving files, proxying, redirects, caching, and more.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

7. NGINX as a Reverse Proxy

1. What is a Reverse Proxy?

- A reverse proxy sits between the client and your application server.
- Clients send requests to NGINX, and NGINX forwards them to the application.
- The client does not connect directly to the application server.
- It helps with security, load balancing, SSL termination, and more.

2. Request Flow (Client → NGINX → Application)



3. Basic Reverse Proxy Configuration

A simple example that proxies requests to an application running on localhost port 3000.

```

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
  
```

4. Important Directives

Directive	Purpose
<code>proxy_pass</code>	Defines the backend application URL.
<code>proxy_set_header Host \$host</code>	Sends the original Host header.
<code>proxy_set_header X-Real-IP \$remote_addr</code>	Passes the client's real IP address.
<code>proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for</code>	Passes the client's IP chain through proxies.
<code>proxy_set_header X-Forwarded-Proto \$scheme</code>	Passes the original protocol (http or https).
<code>proxy_http_version 1.1</code>	Uses HTTP/1.1 for the connection to the backend.
<code>proxy_set_header Connection ""</code>	Prevents connection issues with some applications.

5. Testing the Reverse Proxy

Start your application on port 3000 (example: Node.js).

```

# Example: Start a Node.js app
node app.js
# App should be running on http://localhost:3000
  
```

Access it through NGINX.

<http://example.com>

You should see the application response in your browser.

6. Understanding Proxy Headers

These headers provide important information to the backend.

Header	What It Contains
<code>Host</code>	The original domain name requested by the client.
<code>X-Real-IP</code>	The real IP address of the client.
<code>X-Forwarded-For</code>	A list of IP addresses of all proxies in the chain.
<code>X-Forwarded-Proto</code>	The protocol used by the client (http or https).

8. Common Reverse Proxy Scenarios

Scenario	Example proxy_pass
Proxy to local app	<code>proxy_pass http://localhost:3000;</code>
Proxy to another server	<code>proxy_pass http://10.0.0.5:3000;</code>
Proxy to a domain	<code>proxy_pass http://api.example.com;</code>
Proxy with a path	<code>proxy_pass http://localhost:3000/api/;</code>
Proxy multiple apps	Use different location blocks for each app.

7. Exposing an Application Safely

- Keep the application server on localhost or a private network.
- Only expose NGINX to the internet.
- Use firewall rules to allow ports 80/443 only on the NGINX server.
- Add SSL (HTTPS) for secure access.
- You can also add authentication, rate limiting, and other security measures.



Junior Engineer Tip

A reverse proxy hides your application server, improves security, and gives you control over how traffic is handled.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

8. Proxy a Real Application

1. Start an Application on localhost:3000

We will use a simple Node.js application as an example.

```
# Create a simple Node.js app
mkdir my-app && cd my-app
npm init -y
npm install express

# Create app.js
cat > app.js <<'EOF'
const express = require('express');
const app = express();
app.get('/', (req, res) => {
  res.send('Hello from Node.js app!');
});
app.listen(3000, () => {
  console.log('App running on http://localhost:3000');
});
EOF
```

Start the application.

```
node app.js
# App running on http://localhost:3000
```

2. Architecture Overview

Client accesses the application through NGINX instead of directly.



3. Configure NGINX as a Reverse Proxy

Create a server block to proxy requests to the application.

`/etc/nginx/sites-available/my-app.conf`

```
server {
  listen 80;
  server_name example.com; # use your domain or server IP

  location / {
    proxy_pass http://localhost:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

4. Enable the Configuration

Create a symbolic link and test the configuration.

```
sudo ln -s /etc/nginx/sites-available/
my-app.conf /etc/nginx/sites-enabled/

sudo nginx -t
# syntax is ok

sudo systemctl reload nginx
```

5. Test with curl

Send a request to NGINX and check the response.

```
curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: text/html; charset=utf-8
Content-Length: 25
Connection: keep-alive
```

6. Test in a Browser

Open your server IP or domain in a browser.



7. Verify the Complete Request Path

Check that the request goes through NGINX to the backend.

```
# 1. Check NGINX access log
sudo tail -f /var/log/nginx/access.log
192.168.1.10 - - [01/Oct/2026:10:15:30 +0000] "GET / HTTP/1.1" 200 -

# 2. Check the Node.js application log
# (you should see the request hit the app)
App running on http://localhost:3000
GET / 200 12.345 ms - 25
```



Junior Engineer Tip

- The application runs on localhost and is not exposed to the internet.
- NGINX handles incoming traffic and proxies it to the application.
- You can use a domain name, add SSL later, and apply security settings.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

9. NGINX Headers Explained

1. What are Proxy Headers?

- Proxy headers carry important information from the client to the backend application through NGINX.
- They tell the application who the real client is, what protocol was used, and other request details.
- Without these headers, the application may only see the IP address of NGINX instead of the actual client.

2. Request Flow with Headers



Original Request Headers (from client)

Host: example.com
 User-Agent: ...
 Accept: ...
 X-Forwarded-For: (added by NGINX)

Request Headers (to backend app)

Host: example.com
 X-Real-IP: 203.0.113.10
 X-Forwarded-For: 203.0.113.10
 X-Forwarded-Proto: https

3. Common NGINX Proxy Headers

Header	What It Contains
Host	The original domain name requested by the client.
X-Real-IP	The real IP address of the client.
X-Forwarded-For	A list of IP addresses of all proxies in the chain (client IP first).
X-Forwarded-Proto	The protocol used by the client (http or https).
X-Forwarded-Host	The original host requested by the client.
X-Forwarded-Port	The original port used by the client.

4. NGINX Configuration with Proxy Headers

Add these headers in your server block when proxying to an application.

```

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Port $server_port;
    }
}
  
```

5. Example: Headers Seen by the Application

When a client with IP 203.0.113.10 accesses your site through NGINX, your application will see:

```

curl -I http://example.com
HTTP/1.1 200 OK
Server: nginx/1.24.0
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
X-Real-IP: 203.0.113.10
X-Forwarded-For: 203.0.113.10
X-Forwarded-Proto: http
X-Forwarded-Host: example.com
X-Forwarded-Port: 80
  
```

6. Why Applications Need These Headers

- To identify the real client IP address.
- To generate correct redirects (http to https).
- To build absolute URLs with the correct domain and protocol.
- To apply security rules, rate limiting, and logging based on the real client IP.
- To work correctly behind a reverse proxy or load balancer.

7. What Information Moves Through a Proxy?



Junior Engineer Tip

- Always set proxy headers when using NGINX as a reverse proxy.
- Use \$proxy_add_x_forwarded_for to preserve the full client IP chain.
- Applications will only see the real client IP if these headers are configured.
- These headers are important for logging, security, and correct application behavior.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



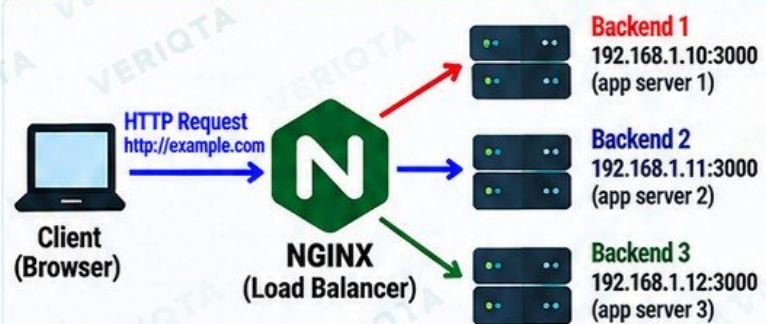
@veriqta

10. NGINX Load Balancing

1. What is NGINX Load Balancing?

- NGINX can distribute incoming requests across multiple backend servers.
- It improves availability, scalability, and fault tolerance.
- It uses the upstream block to define backend servers.
- NGINX supports different load balancing methods and can handle server failures automatically.

2. Three-Backend Architecture (Example)



3. NGINX Upstream Configuration

Define multiple backend servers using the upstream block.

```
upstream my_app {
    server 192.168.1.10:3000;
    server 192.168.1.11:3000;
    server 192.168.1.12:3000;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://my_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

6. Testing the Load Balancer

Start three backend servers (example with Node.js).

```
# Terminal 1 - Backend 1
PORT=3000 node app.js # output: App 1

# Terminal 2 - Backend 2
PORT=3001 node app.js # output: App 2

# Terminal 3 - Backend 3
PORT=3002 node app.js # output: App 3
```

Test the load balancer with curl (multiple requests).

```
for i in {1..6}; do
    curl -s http://example.com
    echo
done
```

You should see responses from App 1, App 2, and App 3
in a round-robin manner.

4. Load Balancing Methods

NGINX supports different load balancing algorithms.

Method	Configuration	How It Works	Use Case
Round Robin (default)	No additional configuration needed.	Distributes requests evenly across all healthy servers.	General use case.
least_conn	least_conn;	Sends requests to the server with the least active connections.	Useful when requests take different time to complete.
ip_hash	ip_hash;	Uses the client's IP address to always send the same client to the same server.	Useful for sessions without a shared session store.

5. Failure Handling

NGINX can automatically handle failed backend servers.

```
upstream my_app {
    server 192.168.1.10:3000 max_fails=3 fail_timeout=30s;
    server 192.168.1.11:3000 max_fails=3 fail_timeout=30s;
    server 192.168.1.12:3000 max_fails=3 fail_timeout=30s;
}
```

- If a server fails (e.g., is unreachable), NGINX stops sending requests to it for the specified time.
- When the server becomes healthy again, NGINX will start sending requests to it.

7. Example Response from Backends

Each backend returns a different response so you can see the load balancing.

```
$ curl http://example.com
Hello from App 1 - 192.168.1.10:3000
$ curl http://example.com
Hello from App 2 - 192.168.1.11:3000
$ curl http://example.com
Hello from App 3 - 192.168.1.12:3000
```



Junior Engineer Tip

- Use health checks, timeouts, and multiple servers for high availability.
- Choose the right load balancing method based on your application.
- Always test with multiple requests to confirm the distribution.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

11. Domains, DNS, and NGINX

1. What are Domains and DNS?

- A domain name (e.g. `example.com`) is a human readable address for your server.
- DNS (Domain Name System) converts the domain name into an IP address.
- DNS records like A and AAAA point your domain to your server's public IP address.
- Once the domain is resolved to an IP, the browser can connect to your NGINX server.

3. Common DNS Records

Record Type	Points To	Purpose	Example
A	IPv4 address	Maps domain to an IPv4 address	<code>example.com</code> → <code>203.0.113.10</code>
AAAA	IPv6 address	Maps domain to an IPv6 address	<code>example.com</code> → <code>2001:db8::10</code>
CNAME	Another domain	Alias to another domain	<code>www</code> → <code>example.com</code>
MX	Mail server	Used for email	<code>example.com</code> → <code>mail.example.com</code>
TXT	Text value	Used for verification, security (e.g. SPF, DKIM)	<code>v=spf1 ...</code>

5. Ports 80 and 443

Port	Protocol	Purpose	Notes
80	HTTP	Unencrypted web traffic	Common for initial access and redirects to HTTPS.
443	HTTPS	Encrypted web traffic (SSL/TLS)	Used for secure communication with a valid SSL certificate.

7. Real-World Example

Domain: `example.com` → Public IP: `203.0.113.10`

```
# /etc/nginx/sites-available/example.com
server {
    listen 80;
    server_name example.com www.example.com;

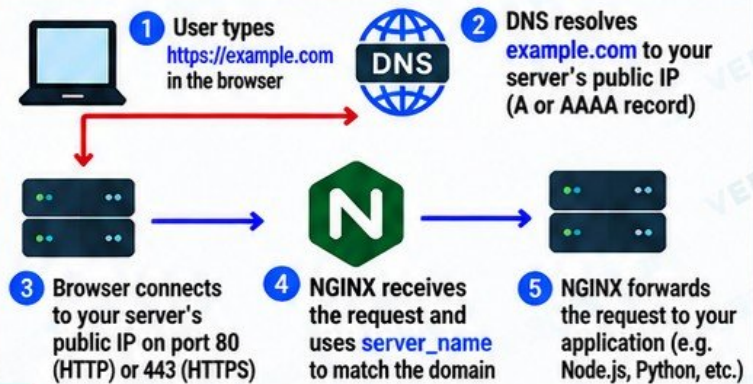
    # Redirect HTTP to HTTPS (optional)
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name example.com www.example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

2. How a Domain Reaches Your NGINX Server



4. NGINX server_name Directive

NGINX uses the `server_name` directive to match the incoming domain.

```
server {
    listen 80;
    server_name example.com www.example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
    }
}
```

6. Complete Request Flow (Detailed)

- 1 User enters `https://example.com` in the browser.
- 2 DNS resolves `example.com` to a public IP (A or AAAA record).
- 3 Browser connects to the server's public IP on port 443 (HTTPS) (or 80 for HTTP).
- 4 NGINX receives the request and matches the domain using `server_name`.
- 5 NGINX processes the request (serves static files or proxies to an application).
- 6 If proxying, NGINX forwards the request to the backend application.
- 7 The application processes the request and sends a response back to NGINX.
- 8 NGINX returns the response to the browser.

8. Testing and Verification

Check DNS Resolution

```
nslookup example.com
# or
dig example.com
```

Check the Resolved IP

```
ping example.com
# or
curl -I http://example.com
```

Check NGINX is Listening

```
sudo ss -tlnp | grep nginx
# or
sudo netstat -tlnp | grep nginx
```

Test the Full Flow

```
curl -I https://example.com
```

- You should see a `HTTP/1.1 200 OK` or a redirect (301) depending on your configuration.



Key Takeaways

- DNS A/AAAA records point your domain to your server's public IP.
- NGINX uses `server_name` to match the domain. Port 80 is for HTTP and port 443 is for HTTPS.
- The browser → DNS → NGINX → application flow enables your domain to reach your application.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



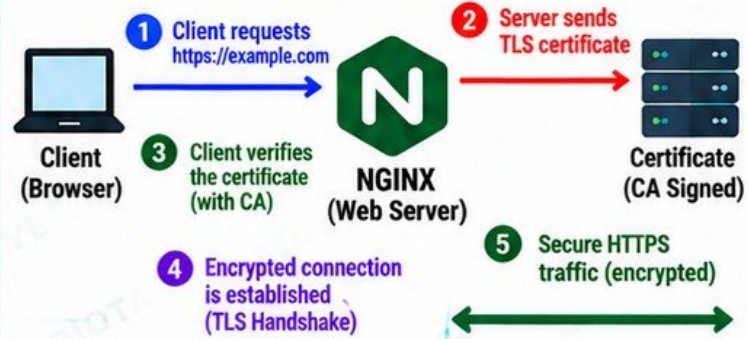
@veriqta

12. HTTPS and TLS with NGINX

1. HTTP vs HTTPS

Feature	HTTP	HTTPS
Protocol	HTTP	HTTP over TLS
Port	80	443
Encryption	No	Yes (TLS)
Data in transit	Readable (not secure)	Encrypted (secure)
Use case	Not recommended for production	Required for production

2. How HTTPS Works (TLS Handshake)



3. TLS Certificate and Key Files



Certificate File
(e.g. example.com.crt)

- Public certificate
- Shared with clients
- Issued by a Certificate Authority (CA) or self-signed



Private Key File
(e.g. example.com.key)

- Private key
- Keep it secret
- Must never be shared
- Used by NGINX to decrypt TLS traffic

5. HTTP to HTTPS Redirect

Redirect all HTTP traffic (port 80) to HTTPS (port 443).

```
server {
    listen 80;
    server_name example.com www.example.com;

    return 301 https://$host$request_uri;
}
```

6. Obtain a Certificate with Let's Encrypt

Use Certbot to get a free TLS certificate.

```
sudo apt update
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com
```

7. Verify HTTPS

Check that your site is using HTTPS and the certificate is valid.

```
# Check with curl
curl -I https://example.com

# Check certificate details
openssl s_client -connect example.com:443 -servername example.com
```

4. NGINX Configuration for HTTPS

Example server block with TLS certificate and key.

```
server {
    listen 443 ssl;
    server_name example.com www.example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

8. Important SSL Directives

Directive	Purpose
<code>listen 443 ssl</code>	Tells NGINX to listen on port 443 with TLS.
<code>ssl_certificate</code>	Path to the TLS certificate (public).
<code>ssl_certificate_key</code>	Path to the private key (keep it secure).
<code>ssl_protocols</code>	Specify allowed TLS versions (e.g. TLSv1.2 TLSv1.3).
<code>ssl_ciphers</code>	Define secure cipher suites.
<code>ssl_prefer_server_ciphers</code>	Prefer server's cipher order.
<code>ssl_session_cache</code>	Enable TLS session caching for better performance.

9. Why Private Keys Must Be Protected



- The private key is used to decrypt TLS traffic.
- If it is exposed, attackers can impersonate your server.
- Keep the private key on the server, with strict file permissions (e.g. `chmod 600`).
- Never share it or commit it to Git repositories.
- Use a trusted CA (e.g. Let's Encrypt, DigiCert, etc.).

Key Takeaways

- HTTPS uses TLS to encrypt traffic between the client and server.
- Use a valid TLS certificate and keep the private key secure.
- Port 80 is for HTTP and should redirect to HTTPS.
- Port 443 is for HTTPS.
- HTTPS protects user data and builds trust with your users.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

13. NGINX Logging

1. What is NGINX Logging?

- NGINX records information about every request and any errors that occur.
- Logs help you monitor, troubleshoot, and understand your server activity.
- The two main logs are `access.log` and `error.log`.

3. Configure NGINX Logging (Example)

Define custom log format and log file paths in NGINX.

```
http {
    log_format main '$remote_addr - $remote_user
    [$time_local] "$request" $status $body_bytes_sent
    "$http_referer" "$http_user_agent";

    access_log /var/log/nginx/access.log main;
    error_log /var/log/nginx/error.log warn;
}
```

5. Reading Logs in Real Time

Use `tail -f` to monitor logs as requests come in.

```
# Monitor access log
sudo tail -f /var/log/nginx/access.log

# Monitor error log
sudo tail -f /var/log/nginx/error.log
```

6. Useful grep Commands

Filter logs to find specific information.

```
# Find requests from a specific IP
grep "203.0.113.10" /var/log/nginx/access.log

# Find a specific status code (e.g. 404)
grep " 404 " /var/log/nginx/access.log

# Find requests to a specific URL
grep "/api" /var/log/nginx/access.log

# Find errors in the error log
grep "error" /var/log/nginx/error.log
```

8. Using Logs for Troubleshooting

- Check `access.log` to see if requests are reaching NGINX.
- Use status codes to identify the problem (e.g. 404, 500).
- Check `error.log` for detailed error messages.
- Combine `grep` and timestamps to trace specific requests.
- Logs provide evidence and help you identify the root cause.

2. NGINX Log Files Location

```
# Default log file locations
/var/log/nginx/access.log # Records all requests
/var/log/nginx/error.log  # Records errors and warnings
```



access.log

- Client IP address
- Request details (method, URL)
- Status code
- Response size
- Referrer and user agent



error.log

- Errors and warnings
- Configuration issues
- Upstream/backend errors
- SSL/TLS errors
- Helpful for troubleshooting

4. Example Access Log Entry

```
203.0.113.10 - - [01/Oct/2026:14:23:15 +0000]
"GET /index.html HTTP/1.1" 200 1256
"https://example.com" "Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 Chrome/118.0.0.0"
```

What Each Part Means

- **203.0.113.10** – Client IP address
- **GET /index.html HTTP/1.1** – Request method and URL
- **200** – HTTP status code (200 = success)
- **1256** – Response size in bytes
- **https://example.com** – Referrer (where the request came from)
- **Mozilla/5.0 ...** – User agent (browser information)

7. Common Status Codes in Access Logs

Status Code	Meaning
200	Success (OK)
301	Moved Permanently (redirect)
302	Found (temporary redirect)
400	Bad Request (client error)
401	Unauthorized
403	Forbidden (access denied)
404	Not Found (resource not found)
500	Internal Server Error (server error)
502	Bad Gateway (upstream error)
503	Service Unavailable (server overloaded)



Key Takeaways

- NGINX logs every request and error.
- `access.log` shows who, what, and the response status.
- `error.log` helps you diagnose issues.
- Use `tail -f`, `grep`, and status codes to analyze logs.
- Logs are essential for troubleshooting and understanding server behavior.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

14. HTTP Status Codes You Will Troubleshoot

1. What Are HTTP Status Codes?

- HTTP status codes are 3-digit numbers that show the result of a client's request.
- They help you understand whether the request succeeded, was redirected, had a client error, or had a server error.
- When troubleshooting with NGINX, status codes give you clues about what is working and what is failing.

2. How a Request Flows (with Status Code)



3. Common HTTP Status Codes and NGINX-Specific Examples

Status Code	Name	Meaning	NGINX-Specific Examples (What It Can Indicate)	Example Output
200	OK	Request succeeded	● NGINX successfully served the file. ● Proxy to backend worked.	<code>\$ curl -I https://example.com</code> HTTP/1.1 200 OK
301	Moved Permanently	Resource permanently redirected	● HTTP to HTTPS redirect. ● Domain redirect (www → non-www). ● URL structure changed.	<code>\$ curl -I http://example.com</code> HTTP/1.1 301 Moved Permanently
302	Found (Temporary)	Resource temporarily redirected	● Temporary redirect (e.g. maintenance). ● App redirecting to another URL.	<code>\$ curl -I https://example.com</code> HTTP/1.1 302 Found
400	Bad Request	Invalid request from client	● Malformed request headers. ● Invalid URL or syntax. ● Request too large.	<code>\$ curl -I "http://example.com/%"</code> HTTP/1.1 400 Bad Request
403	Forbidden	Client is not allowed to access the resource	● File permissions issue. ● Directory listing disabled. ● Access restricted by IP or rules (allow/deny).	<code>\$ curl -I https://example.com/secret</code> HTTP/1.1 403 Forbidden
404	Not Found	Resource does not exist	● Incorrect URL. ● File not found in document root. ● Wrong location block in NGINX.	<code>\$ curl -I https://example.com/unknown</code> HTTP/1.1 404 Not Found
500	Internal Server Error	Generic server error	● Misconfigured NGINX. ● Error in NGINX configuration. ● Permission issues.	<code>\$ curl -I https://example.com</code> HTTP/1.1 500 Internal Server Error
502	Bad Gateway	NGINX received an invalid response from upstream	● Backend application is down. ● Wrong proxy_pass (wrong port). ● Application returned an invalid response.	<code>\$ curl -I https://example.com</code> HTTP/1.1 502 Bad Gateway
503	Service Unavailable	Service is temporarily unavailable	● Backend server is overloaded or down. ● Maintenance mode. ● NGINX configured to return 503.	<code>\$ curl -I https://example.com</code> HTTP/1.1 503 Service Unavailable
504	Gateway Timeout	Upstream server took too long to respond	● Application is slow. ● Backend not responding. ● proxy_read_timeout is too low.	<code>\$ curl -I https://example.com</code> HTTP/1.1 504 Gateway Timeout

4. Troubleshooting with curl

Check the status code of a URL.

```
# Check only the status code
curl -o /dev/null -s -w "%{http_code}\n" https://example.com

# Check full headers
curl -I https://example.com

# Follow redirects and see final status
curl -L -o /dev/null -s -w "%{http_code}\n" http://example.com
```

5. Check NGINX Logs

Use logs to find the real reason for the status code.

```
# View access logs (see status codes)
sudo tail -f /var/log/nginx/access.log

# View error logs (see detailed errors)
sudo tail -f /var/log/nginx/error.log

# Find 404 errors
grep " 404 " /var/log/nginx/access.log

# Find 500 errors
grep " 500 " /var/log/nginx/access.log
```



Key Takeaways

- Status codes tell you exactly what happened to a request.
- 2xx = success, 3xx = redirect, 4xx = client error, 5xx = server error.
- Each status code gives a clue for troubleshooting in NGINX.
- Use curl and NGINX logs to investigate issues.
- Fix the root cause based on the status code and error messages.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

15. Troubleshoot 403 and 404 Errors

1. What Do 403 and 404 Mean?

403 Forbidden

- The server understood the request but refuses to access the resource.
- Usually a permission or configuration issue.
- Common causes: wrong file permissions, directory access denied, missing index file, or restrictive location block.

404 Not Found

- The server cannot find the requested resource.
- Usually a wrong path, missing file, or incorrect document root.
- Common causes: file does not exist, wrong document root, incorrect location block, or wrong URL.

3. Common Causes

Cause	Error	Description
Wrong document root	404	NGINX looking in the wrong directory.
Missing file	404	The requested file does not exist.
Incorrect location block	404	URL not matching the expected path.
Directory listing disabled and no index file	403	No index.html and listing is disabled.
Linux file permissions too restrictive	403	NGINX user cannot read the files.
SELinux/AppArmor blocking	403	Security policy denies access.
Incorrect try_files rule	404	NGINX cannot find the file defined in try_files.
Symlink path broken	404	Link points to a non-existent file.

6. Check Linux Permissions

Ensure NGINX can read the files and directories.

```
# Check directory permissions
ls -ld /var/www/html

# Check file permissions
ls -l /var/www/html/index.html

# If needed, fix permissions
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html
```

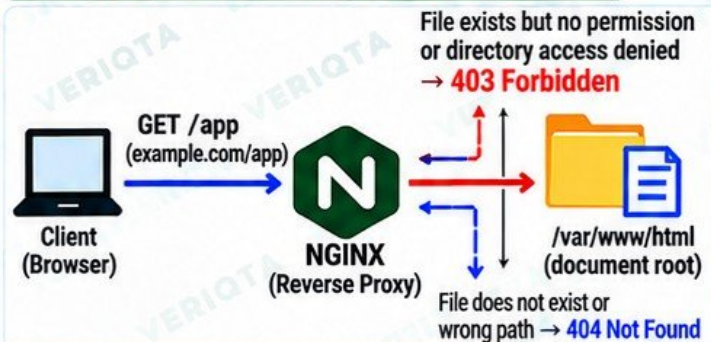


8. Systematic Diagnosis Flow

- 1 Check the requested URL (is the path correct?)
- 2 Verify the document root and location block.
- 3 Confirm the file or directory exists (ls -l).
- 4 Check Linux permissions (chown, chmod).
- 5 Look at the NGINX error log for the exact message.
- 6 Fix the issue and test again (nginx -t && systemctl reload nginx).



2. How a 403 and 404 Happen in NGINX



4. Check the NGINX Configuration

Verify document root and location blocks.

```
server {
    listen 80;
    server_name example.com;

    root /var/www/html; # correct document root
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```



5. Check if the File Actually Exists

Make sure the file is in the correct location.

```
# List files in the document root
ls -l /var/www/html

# Check a specific file
ls -l /var/www/html/index.html

# Check full path resolution
namei -l /var/www/html/index.html
```



7. Check the NGINX Error Log

The error log will give the exact reason.

```
sudo tail -f /var/log/nginx/error.log

# Example 403 error:
[error] 12345#0: *1 open() "/var/www/html/secret/"
failed (13: Permission denied)

# Example 404 error:
[error] 12345#0: *2 open() "/var/www/html/app.html"
failed (2: No such file or directory)
```



9. Real Examples and Fixes

403 Example (Permission Issue)

```
# Fix: give read access
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html
```

404 Example (Wrong Path)

```
# Fix: correct the document root
root /var/www/html; # correct path
# or create the missing file
echo "Hello" > /var/www/html/app.html
```



Key Takeaways

- 403 means the server refuses access, usually a permission issue.
- 404 means the file or resource was not found.
- Always check document root, file existence, and permissions.
- Use ls -l and namei to verify paths and permissions.
- Check the NGINX error log for the exact reason.
- Follow a systematic troubleshooting flow.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

16. Troubleshoot 502 and 504 Errors

1. What Do 502 and 504 Mean?

502 Bad Gateway

- NGINX received an invalid response from the backend server.
- Usually caused by:
 - Backend application down
 - Wrong proxy_pass port
 - Connection refused
 - Application crashing
 - Invalid response from upstream

504 Gateway Timeout

- NGINX could not get a response from the backend server in time.
- Usually caused by:
 - Backend application is slow
 - Application not responding
 - Long-running process
 - Timeout settings too low
 - Network or upstream issues

3. Common Causes and How to Fix

Cause	Error	How to Fix
Backend application is down	502	Start the application and check if it is listening.
Wrong proxy_pass port	502	Verify correct port (e.g. 3000).
Connection refused	502	Check if the app is running and accessible.
Application returns invalid response	502	Check application logs.
Backend application is slow	504	Optimize the application or increase timeout settings.
Long-running process	504	Increase proxy_read_timeout and proxy_connect_timeout.
Network or upstream issue	504	Check server resources, network, or firewall.

6. Check NGINX Configuration

Verify the proxy_pass and timeout settings.

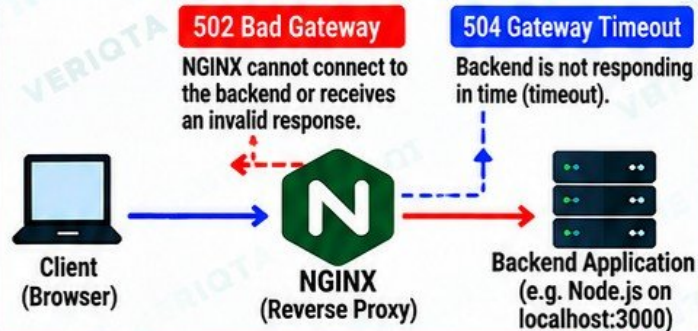
```
server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://localhost:3000; # correct port
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 60s;
        proxy_read_timeout 60s;
        proxy_send_timeout 60s;
    }
}
```

8. Common Error Log Messages

Error Log Message	Meaning
connect() failed (111: Connection refused)	Backend application is not running.
upstream timed out (110: Connection timed out)	Application is not responding in time.
no live upstreams while connecting to upstream	All backend servers are down.
upstream prematurely closed connection	Application closed the connection.
invalid header from upstream	Application returned an invalid response.

2. Architecture (Where Errors Happen)



4. Check if the Backend Application is Running

Make sure your application is running and listening on the correct port.

```
# Check if the application is running (example: Node.js)
systemctl status myapp

# or (if running manually)
ps aux | grep node

# Check if the port is listening
ss -tulnp | grep 3000
# or
netstat -tulnp | grep 3000
```

5. Test the Backend Directly with curl

Test if the application responds without NGINX.

```
# Test local backend application
curl -I http://localhost:3000

# If the application is running, you should see:
HTTP/1.1 200 OK

# If it fails, you may see:
curl: (7) Failed to connect to localhost port 3000: Connection refused
```

7. Investigate the NGINX Error Log

The error log will tell you the exact reason for the 502 or 504 error.

```
# View error logs in real time
sudo tail -f /var/log/nginx/error.log

# Look for 502 errors (connection refused, invalid response)
grep "502" /var/log/nginx/error.log

# Look for 504 errors (upstream timeout)
grep "504" /var/log/nginx/error.log
```

9. Quick Troubleshooting Checklist

- ✓ Is the backend application running?
- ✓ Is it listening on the correct port?
- ✓ Is the proxy_pass URL correct?
- ✓ Can you access the application with curl?
- ✓ Check the NGINX error log for the exact error.
- ✓ Increase timeout values if the application is slow.
- ✓ Check server resources (CPU, memory, network).

Key Takeaways

- 502 means NGINX cannot connect to or gets an invalid response from the backend.
- 504 means the backend did not respond in time.
- Use curl, ss, systemctl and NGINX logs to find the root cause.
- Fix the application, proxy_pass or timeout settings based on the error.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

17. Validate, Reload, and Change NGINX Safely

1. Why You Must Validate Before Applying

- NGINX will not start or reload if the configuration has syntax errors.
- Validating helps you catch mistakes before affecting a live server.
- In production, a small error can cause downtime.
- Professional engineers always test changes before applying them.

2. NGINX Configuration File Location

```
# Main configuration file
/etc/nginx/nginx.conf

# Site configuration files
/etc/nginx/conf.d/
/etc/nginx/sites-available/
/etc/nginx/sites-enabled/
```



NGINX
Configuration

- nginx.conf → main config
- conf.d/ → additional configs
- sites-available/ → site configs
- sites-enabled/ → enabled sites

3. Test the Configuration (Syntax Check)

Use `nginx -t` to check for syntax errors.

```
sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf
syntax is ok
nginx: configuration file /etc/nginx/nginx.conf
test is successful
```

Example of a syntax error:

```
sudo nginx -t
nginx: [emerg] unexpected "}" in /etc/nginx/conf.d/app.conf:12
nginx: configuration file /etc/nginx/nginx.conf test failed
```

5. Reload vs Restart

Use `reload` for most configuration changes.

Command	What It Does	When to Use	Impact
<code>nginx -t</code>	Test configuration syntax	Always before reload or restart	No impact
<code>systemctl reload nginx</code>	Reloads configuration without dropping connections	After successful test (recommended)	No downtime
<code>systemctl restart nginx</code>	Stops and starts NGINX	When reload is not enough	Brief downtime (active connections dropped)

8. Verify the Changes

Check that NGINX is running and the new configuration is active.

```
# Check NGINX status
sudo systemctl status nginx

# Check which configuration file is in use
sudo nginx -T

# Test with curl
curl -I http://localhost
curl -I https://example.com
```



Verify
Changes

4. Backup Configuration Before Changing

Always create a backup before making changes.

```
# Backup main config
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

# Backup a specific site config
sudo cp /etc/nginx/conf.d/app.conf /etc/nginx/conf.d/app.conf.bak

# Create a timestamped backup
sudo cp /etc/nginx/nginx.conf \
/etc/nginx/nginx.conf.$(date +%Y%m%d_%H%M%S)
```

6. Apply Changes Safely (Step by Step)

- 1 Edit the configuration file.
- 2 Test the configuration with `nginx -t`.
- 3 If the test is successful, reload NGINX.
- 4 Verify that the changes are working.
- 5 If something goes wrong, rollback to the backup.



Safe
Change Process

7. Rollback If There Is a Problem

If the new configuration causes issues, restore the backup.

```
# Restore main config
sudo cp /etc/nginx/nginx.conf.bak /etc/nginx/nginx.conf

# Test and reload
sudo nginx -t && sudo systemctl reload nginx
```



Rollback
(Safety Net)

9. Common Mistakes to Avoid

- Skipping `nginx -t` before reloading.
- Editing the wrong file.
- Making changes directly in production without a backup.
- Using restart instead of reload (causes unnecessary downtime).
- Leaving a broken configuration (NGINX will fail to start).
- Not verifying the changes after reloading.



Key Takeaways

- Always test configuration with `nginx -t`.
- Keep a backup before making changes.
- Use reload instead of restart for most changes.
- Have a rollback plan in case something goes wrong.
- Validate and verify changes to prevent downtime.
- Professional engineers test before applying changes, especially in production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

18. NGINX Security Basics

1. Follow the Principle of Least Privilege

- Run NGINX with a non-root user (www-data or nginx).
- Give only the permissions needed.
- Avoid running applications as root.
- Reduce the impact if the service is compromised.



```
# NGINX user (usually set in nginx.conf)
user www-data;
worker_processes auto;
```

3. Use TLS and Strong HTTPS Configuration

- Use HTTPS to encrypt traffic.
- Use a valid TLS certificate (e.g. Let's Encrypt).
- Disable old and insecure TLS versions.
- Use strong cipher suites.



```
server {
    listen 443 ssl;
    server_name example.com;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
}
```

5. Hide Unnecessary Information

- Hide NGINX version information.
- Disable server tokens.
- Remove unnecessary error details.



```
server_tokens off;

# Customize error pages (optional)
error_page 404 /404.html;
error_page 500 502 503 504 /50x.html;
```

7. Request Size and Rate Controls

- Limit client request body size (prevent large uploads).
- Set reasonable timeouts.
- Use rate limiting to prevent abuse.



```
server {
    client_max_body_size 10m; # limit upload size
    client_body_timeout 10s;
    keepalive_timeout 65s;
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
    location / {
        limit_req zone=api burst=20 nodelay;
    }
}
```

9. Additional Good Practices

- Keep NGINX updated.
- Monitor logs for suspicious activity.
- Use a WAF (e.g. ModSecurity) if needed.
- Restrict access to admin endpoints.
- Regularly review your configuration.



2. Secure File and Directory Permissions

- Restrict access to NGINX configuration files.
- Allow only the NGINX user to read required files.
- Keep TLS private keys protected.
- Do not allow world read or write permissions.



```
# Recommended permissions
sudo chown root:root /etc/nginx/nginx.conf
sudo chmod 640 /etc/nginx/nginx.conf
sudo chmod 600 /etc/letsencrypt/live/example.com/privkey.pem
```

4. Add Security Headers

- Security headers help protect your application.
- They reduce common web attacks.
- Add them in your server block.



```
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "no-referrer" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Strict-Transport-Security
    "max-age=63072000; includeSubDomains" always;
```

6. Limit Exposed Services and Ports

- Expose only the required ports (usually 80 and 443).
- Do not expose backend application ports to the internet.
- Use a firewall to block unnecessary ports.



```
# Example UFW rules (only 80 and 443)
sudo ufw allow 80
sudo ufw allow 443
sudo ufw deny 3000 # keep backend private
sudo ufw enable
```

8. Avoid Dangerous Configuration Shortcuts

- Do not use allow all in production.
- Avoid disabling SSL verification.
- Do not use proxy_pass to external URLs without validation.
- Avoid open redirects and insecure default settings.

Examples of bad practices (do not use in production)

```
# Exposes everything (dangerous)
location / {
    allow all;
}

# Disables SSL verification (dangerous)
proxy_ssl_verify off;
```



Key Takeaways

- Use least privilege and secure file permissions.
- Always use HTTPS with strong TLS settings.
- Add security headers.
- Expose only required services and ports.
- Control request sizes and apply rate limiting.
- Avoid insecure shortcuts in your configuration.
- Follow security best practices to reduce risk.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

19. NGINX in Docker and Kubernetes

1. NGINX Container Basics

- NGINX can run as a Docker container.
- It provides the same functionality as a traditional installation.
- Containers make it easy to deploy, scale, and manage NGINX.
- Configuration can be added through a Dockerfile or mounted from the host.



2. Run NGINX in Docker

```
# Pull the official NGINX image
docker pull nginx:latest

# Run NGINX container with port mapping
docker run -d --name nginx-web \
  -p 8080:80 nginx:latest

# Verify the container is running
docker ps

# Test from browser or curl
curl -I http://localhost:8080
```



Access via
http://localhost:8080

3. Mount Custom NGINX Configuration

```
# Create a local nginx.conf
mkdir -p nginx
cp nginx.conf nginx/

# Run container with mounted configuration
docker run -d --name nginx-web \
  -p 8080:80 \
  -v $(pwd)/nginx/nginx.conf:/etc/nginx/nginx.conf \
  -v $(pwd)/nginx/html:/usr/share/nginx/html \
  nginx:latest
```



Mount your
config and
website files

4. Docker NGINX Architecture



5. NGINX in Kubernetes (Overview)

- In Kubernetes, NGINX is usually deployed as a Pod.
- A Service exposes the NGINX Pods inside or outside the cluster.
- An Ingress manages external access, hostnames, and TLS.
- Kubernetes enables high availability, scaling, and zero downtime.

6. Kubernetes NGINX Architecture



7. Example Kubernetes Manifests

Deployment (NGINX)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
```

Service (ClusterIP)

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Ingress (External Access)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
spec:
  rules:
    - host: example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx-service
                port:
                  number: 80
```

8. Key Differences: Traditional vs Containerized

Aspect	Traditional NGINX	Docker	Kubernetes
Installation	Install on server	Run as container	Run as Pod
Configuration	/etc/nginx/nginx.conf	Mount config or use Dockerfile	Use ConfigMap or mounted files
Scaling	Manual	Run multiple containers	Automatic (replicas)
High Availability	Manual setup	Manual (docker-compose)	Built-in (multiple pods and services)
Load Balancing	NGINX upstream	NGINX inside container	Service + Ingress
TLS/Certificates	Install on server	Mount cert files	Use Ingress TLS or cert-manager

9. How Containerized Environments Change the Architecture

- NGINX runs as a container/pod instead of a server process.
- Configuration is externalized using volumes or ConfigMaps.
- Services and Ingress manage traffic instead of direct server IPs.
- Scaling and recovery are automated by Kubernetes.
- It is easier to deploy, update, and roll back changes.



Key Takeaways

- NGINX works the same in Docker and Kubernetes, but the architecture changes.
- Use port mapping in Docker and Services/Ingress in Kubernetes.
- Mount configuration files or use ConfigMaps.
- Containerized environments provide better scalability, high availability, and easier management.



@veriqta



@veriqta



@veriqta



@veriqta



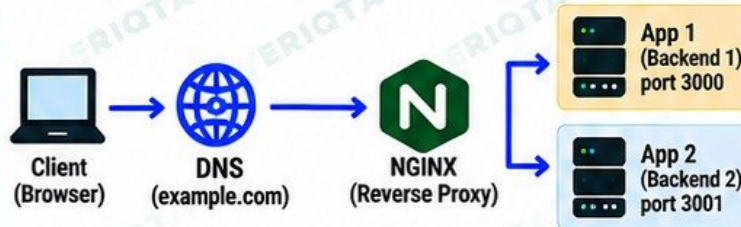
@veriqta



@veriqta

20. Junior Engineer Practical Lab: Deploy and Troubleshoot NGINX

1. Lab Architecture (Mini Production Setup)



3. Configure NGINX as Reverse Proxy with Load Balancing

Edit /etc/nginx/nginx.conf or a site config file.

```

upstream backend_pool {
    server 127.0.0.1:3000;
    server 127.0.0.1:3001;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://backend_pool;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
  
```



Configure reverse proxy and load balancing

2. Set Up Two Application Backends

Create two simple web applications using Docker (Nginx).

```

# Backend 1
docker run -d --name app1 \
  -p 3000:80 \
  nginx:latest

echo "App 1 - Running" \
  > index.html
# (inside container or
# use custom html)
  
```

```

# Backend 2
docker run -d --name app2 \
  -p 3000:80 \
  nginx:latest

echo "App 2 - Running" \
  > index.html
# (inside container or
# use custom html)
  
```

4. Validate and Apply the Configuration

Always test before reloading.

```

# Test configuration
sudo nginx -t

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

# Reload NGINX
sudo systemctl reload nginx

# Check status
sudo systemctl status nginx

# Verify
curl -I http://localhost
curl http://localhost
  
```



5. Test the Application (Load Balancing)

Make multiple requests and see responses from both backends.

```

# Send multiple requests
for i in {1..6}; do
  curl http://localhost;
  echo ""
done
  
```

Example Output (Alternating):

```

App 1 - Running
App 2 - Running
App 1 - Running
App 2 - Running
App 1 - Running
App 2 - Running
  
```

6. Inspect NGINX Logs

View access and error logs.

```

# Access logs (see requests and status codes)
sudo tail -f /var/log/nginx/access.log

# Error logs (see problems)
sudo tail -f /var/log/nginx/error.log
  
```



7. Intentionally Break a Backend

Stop one backend to simulate a failure.

```

# Stop backend 1
docker stop app1

# Or stop backend 2
docker stop app2
  
```



8. Diagnose the Failure

Check logs and test the application.

```

# Test again
curl http://localhost

# Check NGINX error logs
sudo tail -f /var/log/nginx/error.log
  
```

Possible Errors:

- 502 Bad Gateway (if backend is down)
- 504 Gateway Timeout (if backend is slow)
- Connection refused (if port is wrong)

9. Fix the Issue

Start the backend again or fix the configuration.

```

# Start the stopped backend
docker start app1

# If wrong port, update config, test and reload
sudo nginx -t && sudo systemctl reload nginx
  
```



10. Verify the Application is Working

Send multiple requests again.

```

for i in {1..6}; do
  curl http://localhost;
  echo ""
done
  
```

Example Output:

```

App 1 - Running
App 2 - Running
App 1 - Running
App 2 - Running
App 1 - Running
App 2 - Running
  
```



Key Takeaways

- NGINX can reverse proxy and load balance to multiple backends.
- Always test with `nginx -t` before reloading.
- Use logs to diagnose issues.
- Common failures: 502 (backend down), 504 (timeout), connection refused.

- Fix the root cause, validate, reload, and verify the application.
- Load balancing improves availability.
- This lab simulates a real-world production-style setup.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta